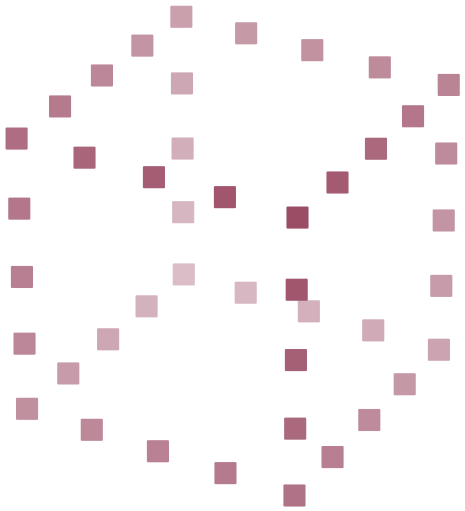
44 cubos en las aristas



Loops and Lerps

Bucles e interpolación en Unity

Eva Gallardo Romero

24 de marzo de 2025

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **2**

4 Requisitos del sistema **3**
4.1 Requisitos funcionales 3
4.2 Requisitos no funcionales 4

5 Documentación técnica **4**
5.1 CubeEdges.cs 4
5.2 TargetList.cs 5
5.3 QuadraticInterpolation.cs y CubicInterpolation.cs 5
5.4 ChangeScene.cs 6

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: Loops-and-lerps.

La documentación del código generada con Doxygen está en evagr11.github.io/Loops-and-lerps.

1 Descripción general

Loops and Lerps es una colección de **cuatro escenas de Unity** hecha en la asignatura de Programación de 1.º de DAM para practicar dos herramientas básicas de los videojuegos: los **bucles** y la **interpolación lineal** (*Lerp*). Se pasa de una escena a la siguiente con la tecla **N**.

1. **CubeEdges**: construye las aristas de un cubo de 5×5×5 con tres bucles `for` anidados, una pieza cada 0,2 segundos.
2. **TargetList**: una esfera recorre una lista de puntos interpolando a la vez su posición, escala, rotación y color.
3. **Cuadrática**: una esfera va y vuelve por una curva de Bézier con un punto de control.
4. **Cúbica**: lo mismo con una curva de Bézier de dos puntos de control (algoritmo de De Casteljau).

Todas las animaciones están hechas por código con **corrutinas**, y el ritmo de cada movimiento se controla con una **AnimationCurve** editable desde el Inspector: con la misma animación, cambiando solo la curva, el objeto acelera, frena o se pasa del destino y vuelve.

2 Diagrama de clases

Cada escena tiene su script. `ChangeScene` es el gestor de escenas: no se destruye al cambiar de escena (`DontDestroyOnLoad`) y guarda la escena actual en la clase estática `SceneData`.

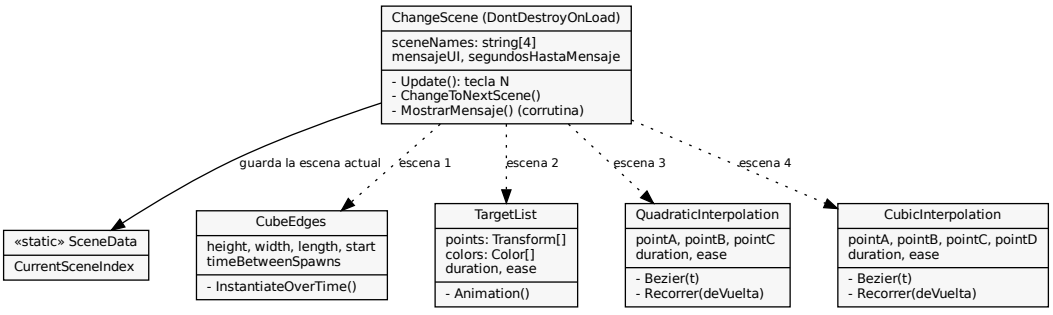


Figura 1: Diagrama de clases de Loops and Lerps

3 Diseño

Las escenas son sencillas para que se vea bien el movimiento: los objetos en el centro y una cámara fija. Si pasan 5 segundos sin cambiar de escena, aparece un mensaje que recuerda que se puede pulsar N.

Las dos figuras siguientes no son capturas del juego: son gráficos hechos con los mismos cálculos que los scripts, para explicar qué dibuja cada escena.



Figura 2: Posiciones que genera CubeEdges con 5×5×5: 44 cubos en las aristas

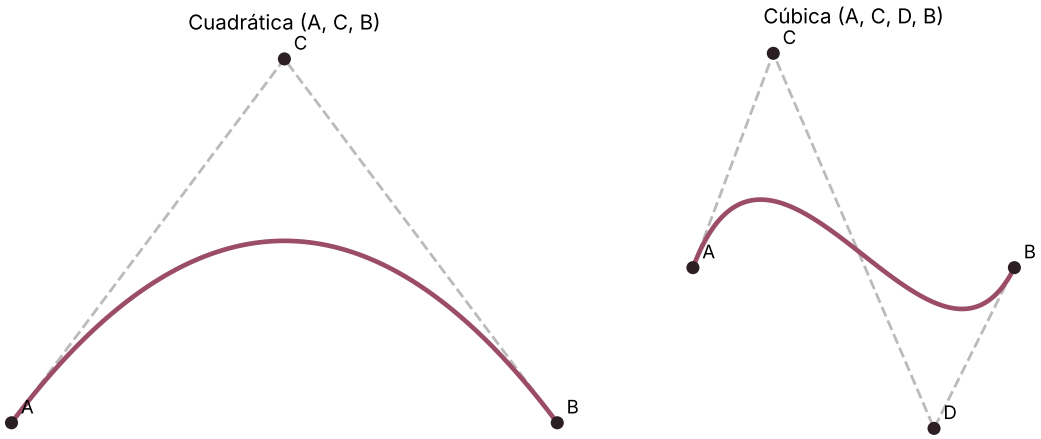


Figura 3: Curvas de Bézier cuadrática y cúbica con sus puntos de control

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	La tecla **N** debe cargar la escena siguiente y volver a la primera después de la última.
RF02	Si no se cambia de escena en 5 segundos, debe aparecer un mensaje de ayuda.
RF03	CubeEdges debe colocar cubos solo en las **aristas** del cubo, uno cada cierto tiempo.

ID	Requisito funcional
RF04	TargetList debe interpolar **posición, escala, rotación y color** entre cada punto y el siguiente, en bucle.
RF05	Las escenas de Bézier deben recorrer la curva de ida y de vuelta sin parar.
RF06	El ritmo de las animaciones debe seguir una **AnimationCurve**.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	El proyecto debe estar hecho en **Unity 2022.3 LTS** con scripts en **C#**.
RNF02	Las animaciones deben hacerse con **corrutinas**, sin el sistema de animación de Unity.
RNF03	Solo puede haber **un gestor de escenas** aunque se vuelva a la primera escena.
RNF04	Los valores (tamaños, tiempos, puntos, colores y curvas) se configuran desde el Inspector.
RNF05	Los scripts deben tener comentarios de documentación `///` para Doxygen.

5 Documentación técnica

Este anexo agrupa la documentación por scripts, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `using` y los comentarios `///`.

5.1 CubeEdges.cs

5.1.1 Ubicación

`Assets/Scripts/CubeEdges.cs`

Descripción general:
Construye las aristas de un cubo poco a poco.

5.1.2 Funciones

InstantiateOverTime **Descripción:** Recorre todas las posiciones con tres bucles y solo pone un cubo donde coinciden al menos dos caras (una arista), esperando entre uno y otro.

Código comentado:

```
IEnumerator InstantiateOverTime()
{
    int endI = start + height - 1;
    int endJ = start + width - 1;
    int endK = start + length - 1;
    for (int i = start; i <= endI; i++)
    {
        for (int j = start; j <= endJ; j++)
        {
            for (int k = start; k <= endK; k++)
            {
                bool bordeI = i == start || i == endI;
                bool bordeJ = j == start || j == endJ;
                bool bordeK = k == start || k == endK;
                // arista: está en el borde de al menos dos ejes
                if ((bordeI && bordeJ) || (bordeJ && bordeK) || (bordeI && bordeK))
                {
                    Instantiate(cube, new Vector3(i, j, k), Quaternion.identity);
```

```
                    yield return new WaitForSeconds(timeBetweenSpawns);
                }
            }
        }
    }
}
```

5.2 TargetList.cs

5.2.1 Ubicación

`Assets/Scripts/TargetList.cs`

Descripción general:
Una esfera que recorre una lista de puntos.

5.2.2 Funciones

Animation **Descripción:** Durante `duration` segundos interpola todo entre el punto actual y el siguiente; después avanza al siguiente y vuelve al primero al acabar la lista.

Código comentado:

```
while (true)
{
    elapsedTime = 0;
    while (elapsedTime < duration)
    {
        elapsedTime += Time.deltaTime;
        normalizedElapsedTime = Mathf.Clamp01(elapsedTime / duration);
        float t = ease.Evaluate(normalizedElapsedTime);     // ritmo de la curva

        // LerpUnclamped: si la curva pasa de 1, la esfera se pasa del destino
        instantiatedObject.position = Vector3.LerpUnclamped(points[currentPoint].
            position, points[nextPoint].position, t);
        instantiatedObject.localScale = Vector3.Lerp(points[currentPoint].localScale,
            points[nextPoint].localScale, t);
        instantiatedObject.rotation = Quaternion.Lerp(points[currentPoint].rotation,
            points[nextPoint].rotation, t);
        instantiatedObjectRenderer.material.color = Color.Lerp(colors[currentPoint],
            colors[nextPoint], t);
        yield return null;
    }
    currentPoint = nextPoint;
    nextPoint = (nextPoint + 1) % points.Length;
}
```

5.3 QuadraticInterpolation.cs y CubicInterpolation.cs

5.3.1 Ubicación

`Assets/Scripts`

Descripción general:
Movimiento por curvas de Bézier calculadas solo con `Lerp`.

5.3.2 Funciones

Bezier (cuadrática) **Descripción:** Dos Lerps hacia el punto de control y un Lerp entre los resultados.

Código comentado:

```
Vector3 Bezier(float t)
{
    Vector3 AC = Vector3.Lerp(pointA.position, pointC.position, t);
    Vector3 CB = Vector3.Lerp(pointC.position, pointB.position, t);
    return Vector3.Lerp(AC, CB, t);
}
```

Bezier (cúbica) **Descripción:** Algoritmo de De Casteljau: tres Lerps, luego dos y por último uno.

Código comentado:

```
Vector3 Bezier(float t)
{
    Vector3 AC = Vector3.Lerp(pointA.position, pointC.position, t);
    Vector3 CD = Vector3.Lerp(pointC.position, pointD.position, t);
    Vector3 DB = Vector3.Lerp(pointD.position, pointB.position, t);
    Vector3 AC_CD = Vector3.Lerp(AC, CD, t);
    Vector3 CD_DB = Vector3.Lerp(CD, DB, t);
    return Vector3.Lerp(AC_CD, CD_DB, t);
}
```

Animation y Recorrer **Descripción:** Recorre la curva de ida y luego de vuelta, usando `1 - t` para ir hacia atrás.

Código comentado:

```
IEnumerator Animation()
{
    while (true)
    {
        yield return Recorrer(false);       // de A a B
        yield return Recorrer(true);        // de B a A
    }
}

IEnumerator Recorrer(bool deVuelta)
{
    float elapsedTime = 0f;
    while (elapsedTime < duration)
    {
        elapsedTime += Time.deltaTime;
        float t = ease.Evaluate(Mathf.Clamp01(elapsedTime / duration));
        instantiatedObject.position = Bezier(deVuelta ? 1f - t : t);
        yield return null;
    }
}
```

5.4 ChangeScene.cs

5.4.1 Ubicación

`Assets/Scripts/ChangeScene.cs`

Descripción general:
Gestor de escenas que sobrevive al cambio de escena.

5.4.2 Funciones

Awake **Descripción:** Si ya hay otro gestor, este se destruye; si no, se conserva entre escenas.

Código comentado:

```
void Awake()
{
    if (FindObjectsOfType<ChangeScene>().Length > 1)
    {
        enabled = false;            // que no llegue a ejecutar Start ni Update
```

```
6           Destroy(gameObject);
7           return;
8       }
9       DontDestroyOnLoad(gameObject);
10      if (SceneData.CurrentSceneIndex == -1)
11          SceneData.CurrentSceneIndex = 0;
12  }
```

ChangeToNextScene y ReiniciarMensaje **Descripción:** Carga la escena siguiente en bucle y vuelve a programar el mensaje de ayuda, parando el anterior.

Código comentado:

```
1  void ChangeToNextScene()
2  {
3      ReiniciarMensaje();
4      SceneData.CurrentSceneIndex = (SceneData.CurrentSceneIndex + 1) % sceneNames.
           Length;
5      SceneManager.LoadScene(sceneNames[SceneData.CurrentSceneIndex]);
6  }
7
8  void ReiniciarMensaje()
9  {
10     mensajeUI.gameObject.SetActive(false);
11     if (mensajeCoroutine != null)
12         StopCoroutine(mensajeCoroutine);
13     mensajeCoroutine = StartCoroutine(MostrarMensaje());   // espera 5 s y lo muestra
14 }
```